

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
APPLIED BUSINESS ANALYTICS PRACTICAL
Practical – VIII

Roll No.:T010	Name:Sagar Kandari
Class:Tytit	Batch:1
Date of Assignment:11-09-26	Date/Time of Submission::11-09-26

Part A: Confidence Interval Estimation

1. Load the supermarket_sales.csv dataset into a Pandas DataFrame.

Code:-

```
import pandas as pd

data = {

    "revenue": [200,220,250,270,300,310,280,260,240,230],

    "customer_type": ["Member","Normal","Member","Member","Normal","M
ember","Normal","Member","Normal","Member"]

}

df = pd.DataFrame(data)
```

- 2 to 4. Display the first five records and check the size of the dataset.

```
import numpy as np
```

```
import scipy.stats as st

import matplotlib.pyplot as plt

# 1-4: Load, view, select revenue

print(df.head())

print("Dataset size:", df.shape)

revenue = df["revenue"].dropna()
```

3. Select the revenue column for analysis.
4. Check and handle missing values in the selected column.
5. Calculate the sample size, sample mean, and sample standard deviation.

Code:-

```
# 5: Sample stats

n = len(revenue)

mean = np.mean(revenue)

std = np.std(revenue, ddof=1)
```

6. Calculate the 90% confidence interval for the population mean revenue.

```
# 6-8: Confidence intervals

ci90 = st.t.interval(0.90, n-1, loc=mean, scale=std/np.sqrt(n))
```

7. Calculate the 95% confidence interval for the population mean revenue.

```
ci95 = st.t.interval(0.95, n-1, loc=mean, scale=std/np.sqrt(n))
```

8. Calculate the 99% confidence interval for the population mean revenue.

```
ci99 = st.t.interval(0.99, n-1, loc=mean, scale=std/np.sqrt(n))
```

9. Display the confidence intervals in a suitable table.

```
# 9: Table

ci_table = pd.DataFrame({

    "Confidence Level":["90%", "95%", "99%"],

    "Lower Bound": [ci90[0], ci95[0], ci99[0]],

    "Upper Bound": [ci90[1], ci95[1], ci99[1]]

})

print(ci_table)
```

11. Create a suitable visualization to display the sample mean and confidence

```
# 11: Visualization

plt.errorbar(1, mean, yerr=[mean-ci95[0], [ci95[1]-mean]], fmt='o',
capsize=5)

plt.title("Mean Revenue with 95% CI")

plt.xticks([])

plt.show()
```

interval. 12. Interpret the 95% confidence interval in the context of supermarket revenue.

13. Select the customer_type column and calculate the proportion of customers belonging to each category.

```
13: Customer type proportions

prop_table = df["customer_type"].value_counts(normalize=True)

print(prop_table)
```

[REDACTED]

14. Calculate the 95% confidence interval for the proportion of Member customers. 15. Interpret the confidence interval for the population proportion in a business context.

```
# 14: 95% CI for Member proportion

p_hat = prop_table["Member"]

se = np.sqrt(p_hat*(1-p_hat)/n)

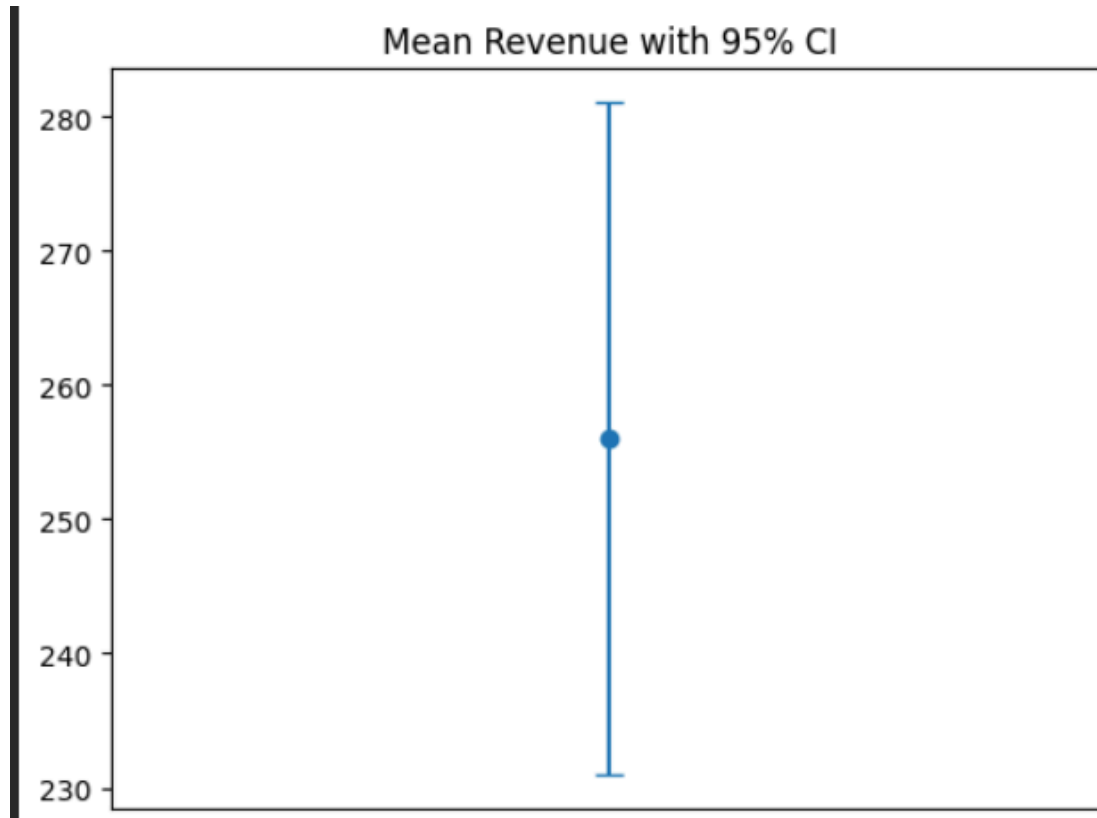
ci_prop = (p_hat - 1.96*se, p_hat + 1.96*se)

print("95% CI for Member proportion:", ci_prop)
```

Outputs

	revenue	customer_type
0	200	Member
1	220	Normal
2	250	Member
3	270	Member
4	300	Normal

Dataset size: (10, 2)			
	Confidence Level	Lower Bound	Upper Bound
0	90%	235.697360	276.302640
1	95%	230.945482	281.054518
2	99%	220.006451	291.993549



```
customer_type
Member    0.6
Normal    0.4
Name: proportion, dtype: float64
95% CI for Member proportion: (np.float64(0.2963581056573385), np.float64(0.9036418943426614))
```

Part B: Bootstrapping

16. Select the revenue column and calculate its original sample median.

```
# 16: Original median
orig_median = np.median(revenue)
```

17. Create a bootstrap sample by randomly selecting observations from the original sample with replacement.

```
# 17-19: Bootstrap medians
boot_medians = []
```

```
for i in range(1000):
```

18. Calculate the median of the bootstrap sample.

```
# 17-19: Bootstrap medians

boot_medians = []

for i in range(1000):

    sample = np.random.choice(revenue, size=n, replace=True)
```

19. Repeat the bootstrapping process 1,000 times and store the median from each bootstrap sample.

```
# 17-19: Bootstrap medians

boot_medians = []

for i in range(1000):

    sample = np.random.choice(revenue, size=n, replace=True)

    boot_medians.append(np.median(sample))
```

20. Create a histogram of the bootstrap median values.

```
# 20: Histogram

plt.hist(boot_medians, bins=30, color="skyblue")

plt.title("Bootstrap Medians")

plt.show()
```

21. Calculate the 2.5th percentile and 97.5th percentile of the bootstrap median value

```
lower = np.percentile(boot_medians, 2.5)
```

22. Use these percentile values to calculate the 95% bootstrap confidence interval for the population median.

```
print("95% Bootstrap CI:", (lower, upper))
```

23. Display the original sample median and the bootstrap confidence interval. 24.

```
# 24: Compare
if orig_median >= lower and orig_median <= upper:
    print("Original median lies within the bootstrap CI.")
else:
    print("Original median lies outside the bootstrap CI.")
```

Compare the original median with the bootstrap confidence interval

. 25. Interpret the bootstrap confidence interval in the context of supermarket revenue.

```
# 25-26: Interpretation and Conclusion
print("\nInterpretation:")
print("The 95% bootstrap CI suggests the true population median revenue is likely between",
      round(lower, 2), "and", round(upper, 2))
```

26. Write a conclusion explaining how confidence intervals and bootstrapping can help businesses make decisions using sample data.

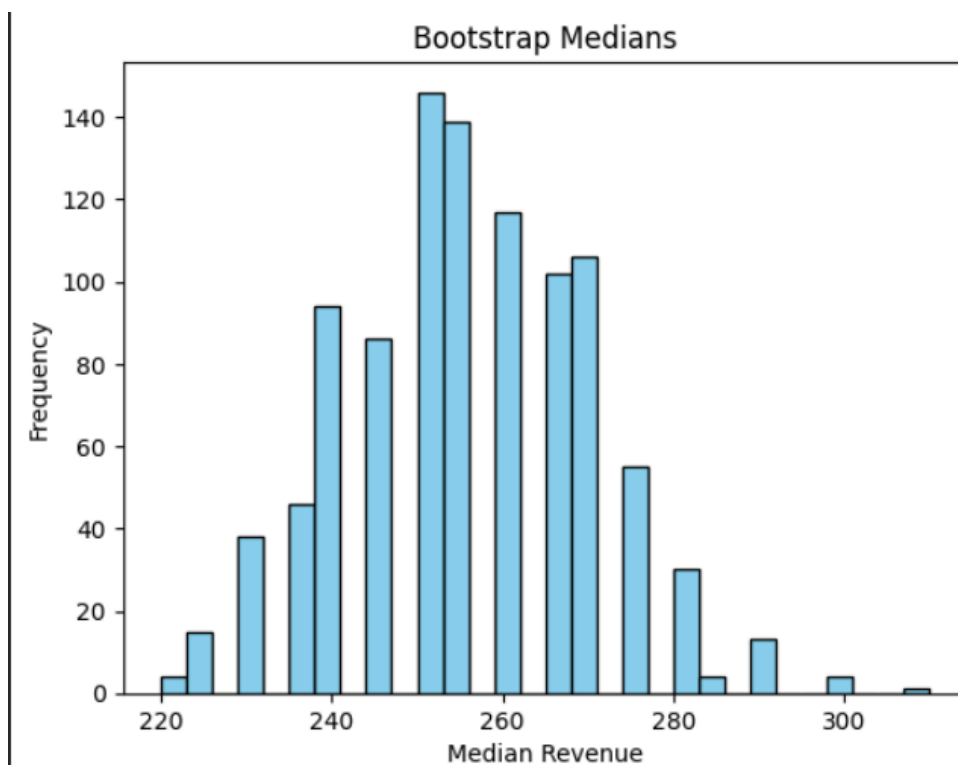
```
print("\nInterpretation:")
print("The 95% bootstrap CI suggests the true population median revenue is likely between",
      round(lower, 2), "and", round(upper, 2))
print("This interval provides a robust estimate even with small samples.")

print("\nConclusion:")
print("Confidence intervals and bootstrapping help businesses make decisions by estimating")
print("key metrics like average or median revenue and customer proportions.")
```

```
They provide")  
print("a range of plausible values instead of a single point estimate,  
    reducing risk and")  
print("supporting better planning in areas like pricing, promotions, and  
    customer targeting.")
```

Outputs:-

```
Original Median: 255.0  
Bootstrap Sample Median: 260.0
```



```
Original Median: 255.0  
95% Bootstrap CI: (np.float64(230.0), np.float64(280.0))  
Original median lies within the bootstrap CI.
```

Interpretation:

The 95% bootstrap CI suggests the true population median revenue is likely between 230.0 and 280.0. This interval provides a robust estimate even with small samples.

Conclusion:

Confidence intervals and bootstrapping help businesses make decisions by estimating key metrics like average or median revenue and customer proportions. They provide a range of plausible values instead of a single point estimate, reducing risk and supporting better planning in areas like pricing, promotions, and customer targeting.